Projeto: Métodos de Ordenação

Lucas Nascimento Aguiar – RA: 10766622

Introdução

Este relatório apresenta os resultados de um estudo prático sobre o tempo de execução de diferentes algoritmos de ordenação. O objetivo central foi comparar o desempenho dos algoritmos Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort em diferentes cenários de entrada (vetores aleatórios, ordenados em ordem crescente e ordenados em ordem decrescente) e tamanhos (1.000, 10.000 e 100.000 elementos).

O projeto foi implementado em linguagem Go e pode ser acessado através do repositório público no GitHub, disponível em: https://github.com/olucasaguiar/projeto-metodos-de-ordenacao.git

Esse repositório contém o código-fonte dos algoritmos, scripts de geração e manipulação dos vetores de teste, além dos relatórios e gráficos produzidos a partir da execução experimental. Dessa forma, o leitor pode acessar diretamente os dados e reproduzir os experimentos, garantindo transparência e reprodutibilidade dos resultados.

Metodologia

A metodologia adotada neste estudo foi estruturada em seis seções principais: ambiente de simulação, linguagem utilizada, implementação dos algoritmos, preparação dos dados, cenários de teste e modo de execução. Essa organização buscou garantir rigor experimental, reprodutibilidade e clareza na análise comparativa dos resultados.

1. Ambiente de Simulação

Os experimentos foram realizados em ambiente controlado, utilizando um computador pessoal com CPU AMD Ryzen 5 3600X (6-Core, 12 threads), memória RAM de 7,7 GiB e sistema operacional Linux (WSL2). Esse ambiente foi selecionado por se tratar de um equipamento pessoal comum e acessível para replicabilidade do experimento. Apesar de se tratar de um computador pessoal, a configuração ainda oferece estabilidade e capacidade de processamento adequadas para a execução dos algoritmos para os volumes de dados do experimento.

2. Linguagem Utilizada

A linguagem escolhida foi Go (versão 1.26.1), devido à sua eficiência na manipulação de memória, suporte nativo a concorrência e precisão nas medições de tempo. A clareza sintática e a facilidade de documentação também foram fatores determinantes para sua adoção neste projeto.

3. Implementação dos Algoritmos

Foram implementados cinco algoritmos clássicos de ordenação: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort. Cada algoritmo foi desenvolvido em módulos independentes, seguindo boas práticas de programação e documentação da linguagem Go. Antes da execução dos testes, todas as implementações foram validadas quanto à correção funcional, assegurando que os resultados refletissem apenas diferenças de desempenho e não erros de implementação.

4. Preparação dos Dados

Os vetores utilizados nos experimentos foram gerados por meio de laços sequenciais, garantindo elementos únicos e sem repetição. Para o cenário embaralhado, foi aplicado o algoritmo Fisher-Yates, que assegura distribuição uniforme das permutações, garantindo que todos os elementos tenham probabilidade equivalente de serem reposicionados. Essa preparação buscou simular condições iniciais distintas de ordenação, permitindo avaliar o impacto da estrutura dos dados sobre o desempenho dos algoritmos.

5. Cenários de Teste

Foram definidos três cenários distintos de entrada, cada um com vetores de tamanhos 1.000, 10.000 e 100.000 elementos:

- Vetores ordenados em ordem crescente.
- Vetores ordenados em ordem decrescente.
- Vetores embaralhados aleatoriamente.

Essa diversidade de cenários permitiu avaliar o comportamento dos algoritmos em condições favoráveis (dados já ordenados), desfavoráveis (dados em ordem inversa) e neutras (dados aleatórios).

6. Modo de Execução

Cada combinação de algoritmo e cenário foi executada cinco vezes consecutivas, com o objetivo de reduzir interferências externas do ambiente de execução. Os tempos de execução foram medidos em nanossegundos, utilizando funções nativas da linguagem Go. Para análise, foram calculadas as métricas de melhor caso, pior caso, média e mediana. Os resultados foram organizados em tabelas e gráficos comparativos, servindo de base para a análise crítica da eficiência prática dos algoritmos frente às suas complexidades assintóticas teóricas.

Resultados e Discussão

A análise experimental dos algoritmos de ordenação foi conduzida em diferentes cenários de entrada e tamanhos de vetor, permitindo uma avaliação crítica da eficiência prática em comparação com a análise assintótica teórica. Os gráficos apresentados a seguir sintetizam os resultados e embasam a discussão.

Visão geral dos tempos médios

O mapa de calor dos tempos médios (Figura 1) fornece uma visão integrada dos resultados. Observa-se que algoritmos quadráticos (Bubble Sort, Selection Sort e Insertion Sort) apresentam tempos de execução que crescem rapidamente com o aumento do tamanho da entrada, chegando a valores da ordem de milhares de segundos para 100.000 elementos. Em contraste, Merge Sort e Quick Sort mantêm tempos significativamente menores, confirmando sua escalabilidade superior.

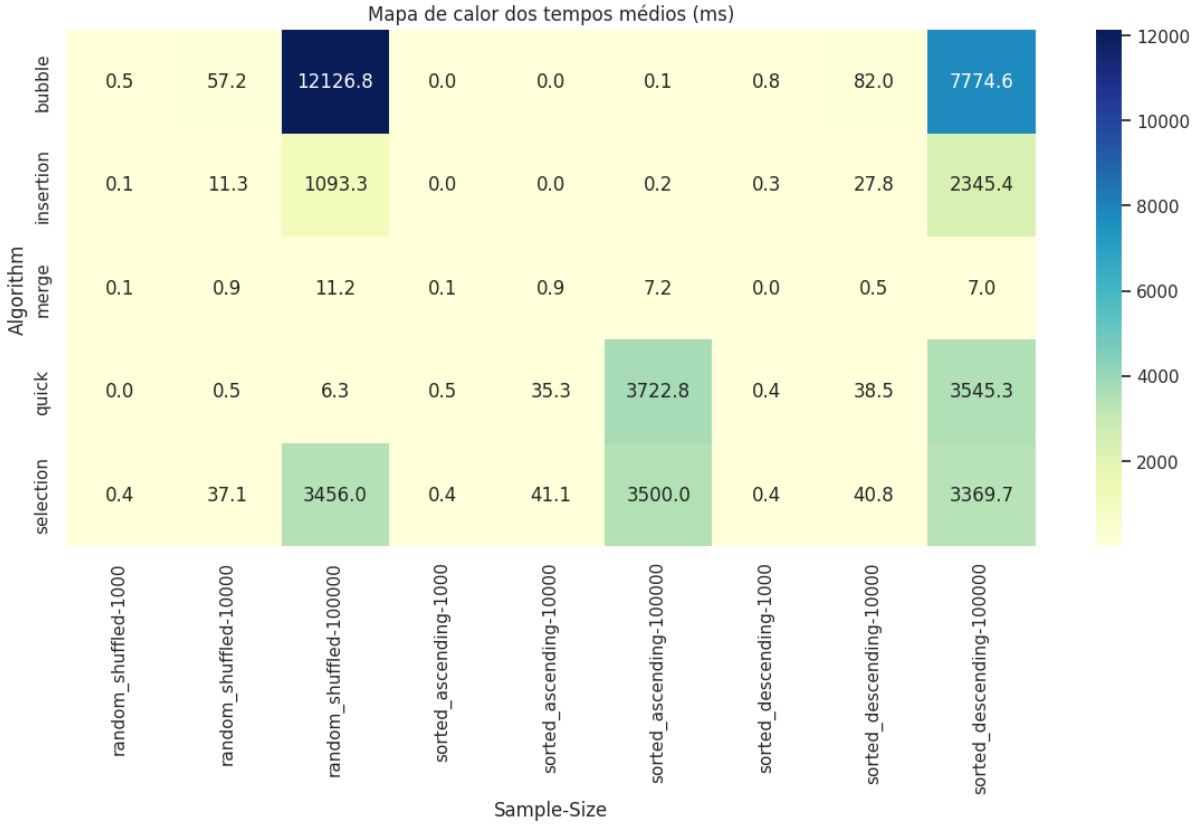

Figura 1 – Mapa de calor dos tempos médios de execução dos algoritmos.

Caso médio

No caso médio (Figura 2), Quick Sort e Merge Sort destacam-se como os algoritmos mais eficientes, mesmo em entradas grandes. Insertion Sort apresenta desempenho competitivo apenas em vetores pequenos, enquanto Bubble Sort e Selection Sort tornam-se inviáveis para grandes volumes de dados. A escala logarítmica utilizada evidencia a diferença de ordem de grandeza entre algoritmos (O(n^2)) e (O(n log n)).

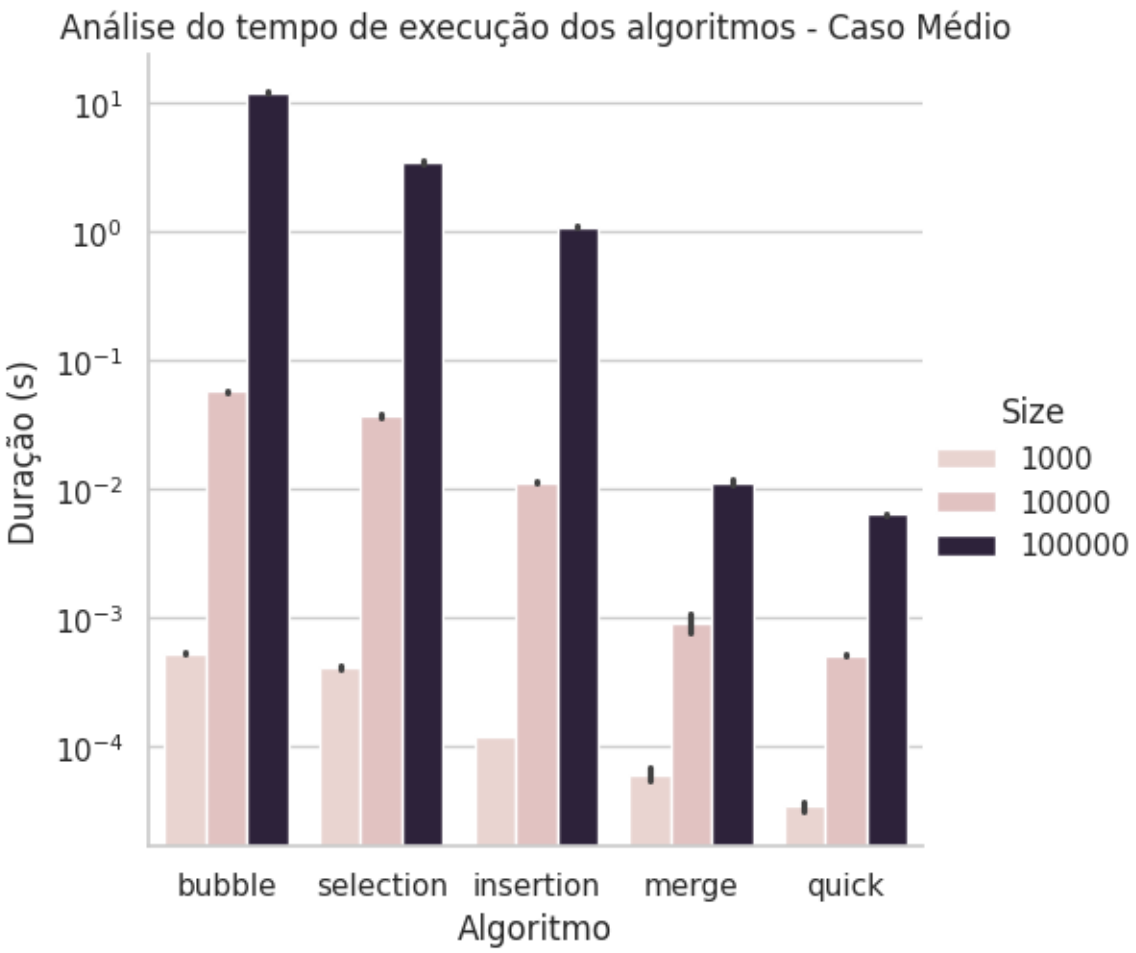

Figura 2 – Tempo de execução dos algoritmos no caso médio.

Melhor caso

No melhor caso (Figura 3), o Insertion Sort apresenta desempenho próximo ao linear quando os dados já estão ordenados, confirmando sua eficiência em cenários favoráveis. Bubble Sort também melhora nesse contexto, mas ainda não compete com Merge Sort e Quick Sort em escalabilidade. Essa análise demonstra que a estrutura inicial dos dados pode favorecer algoritmos simples em situações específicas.

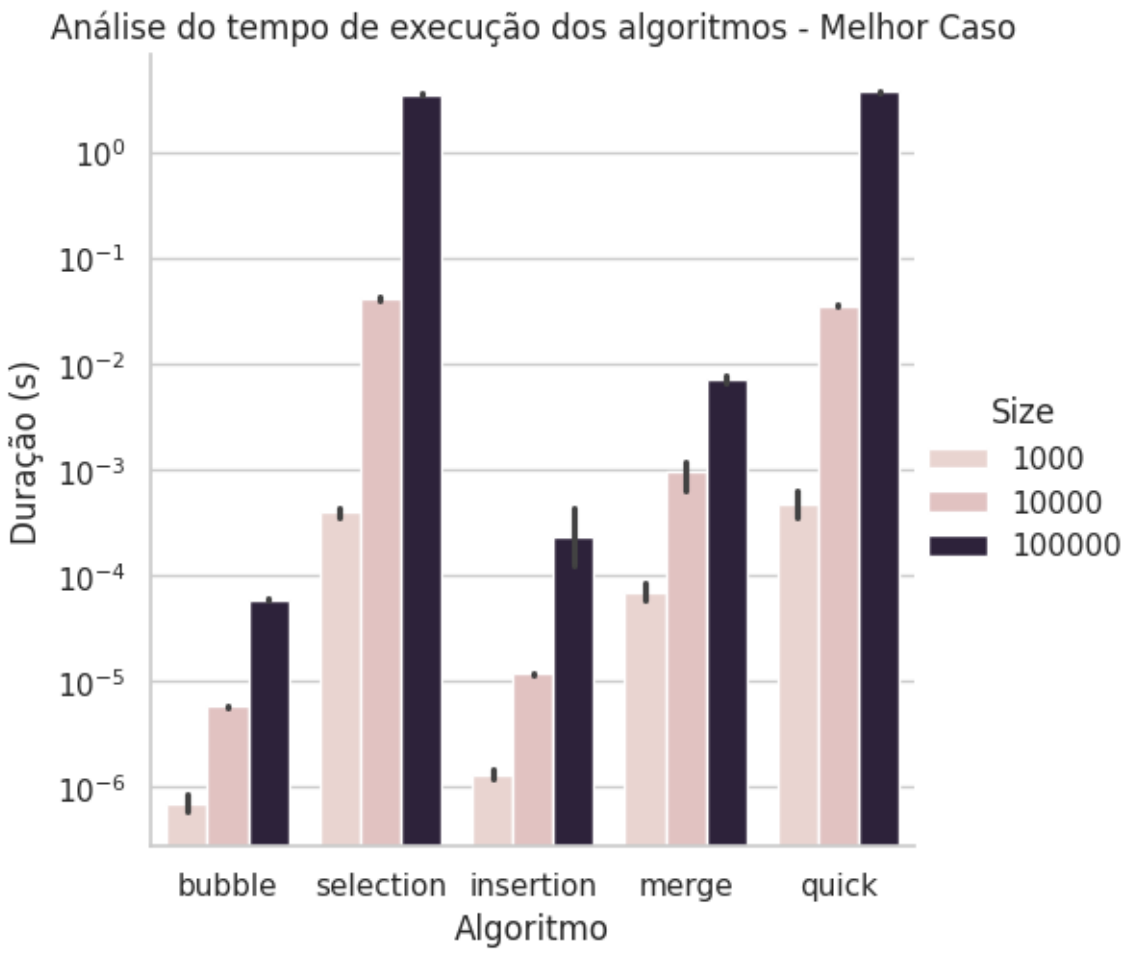

Figura 3 – Tempo de execução dos algoritmos no melhor caso.

Pior caso

No pior caso (Figura 4), a diferença entre os algoritmos é mais acentuada. Bubble Sort, Selection Sort e Insertion Sort apresentam tempos extremamente elevados, inviabilizando seu uso em grandes entradas. Quick Sort sofre degradação significativa em vetores ordenados ou decrescentes, aproximando-se do comportamento quadrático, mas ainda mantém desempenho superior aos algoritmos (O(n^2)). Merge Sort permanece estável, confirmando sua robustez independentemente da configuração inicial.

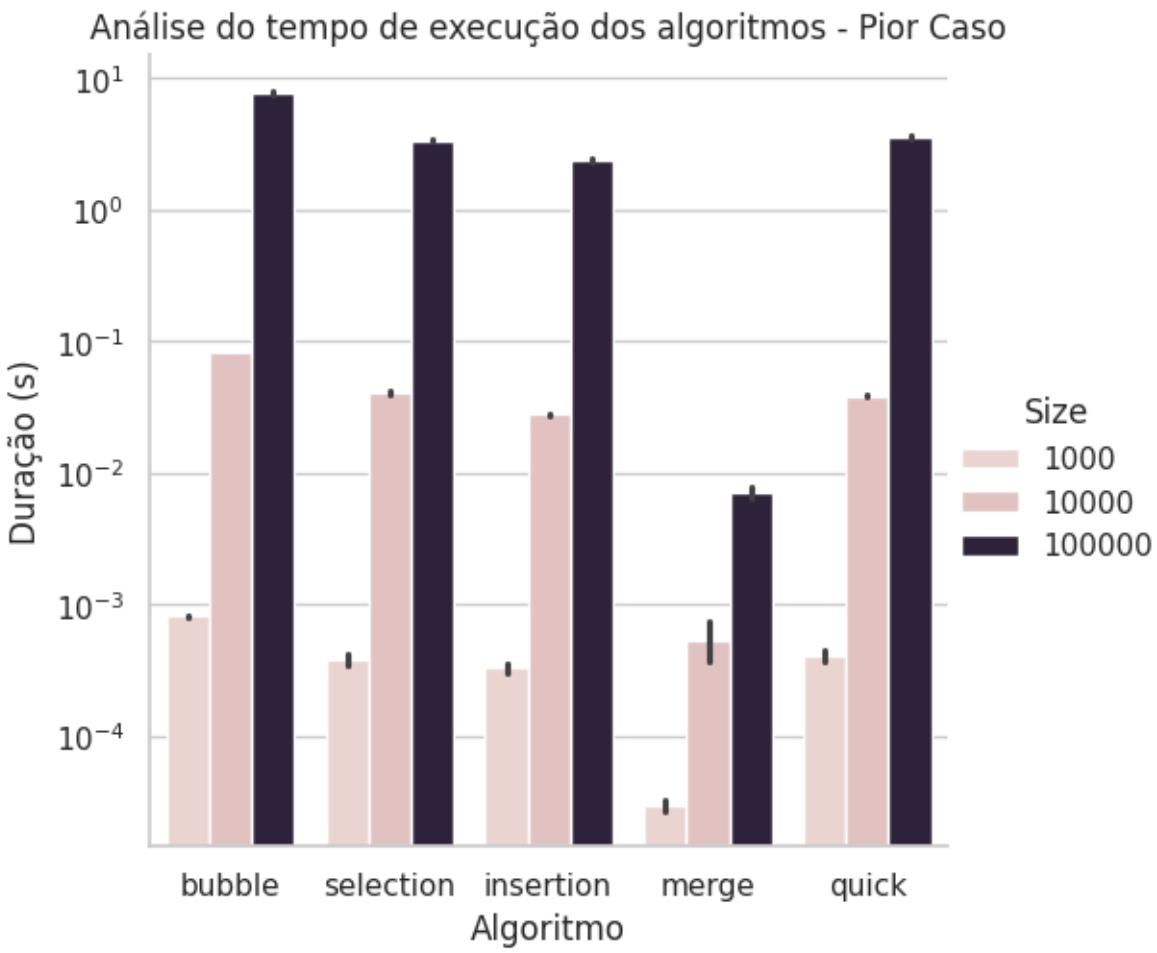

Figura 4 – Tempo de execução dos algoritmos no pior caso.

Comparação por conjunto de dados

A análise segregada por tipo de entrada (Figura 5) mostra como cada algoritmo responde ao ordenamento inicial. Insertion Sort é imbatível em vetores já ordenados, Quick Sort domina em vetores aleatórios e Merge Sort mantém desempenho consistente em todos os cenários. Bubble Sort e Selection Sort são consistentemente inferiores, independentemente da entrada.

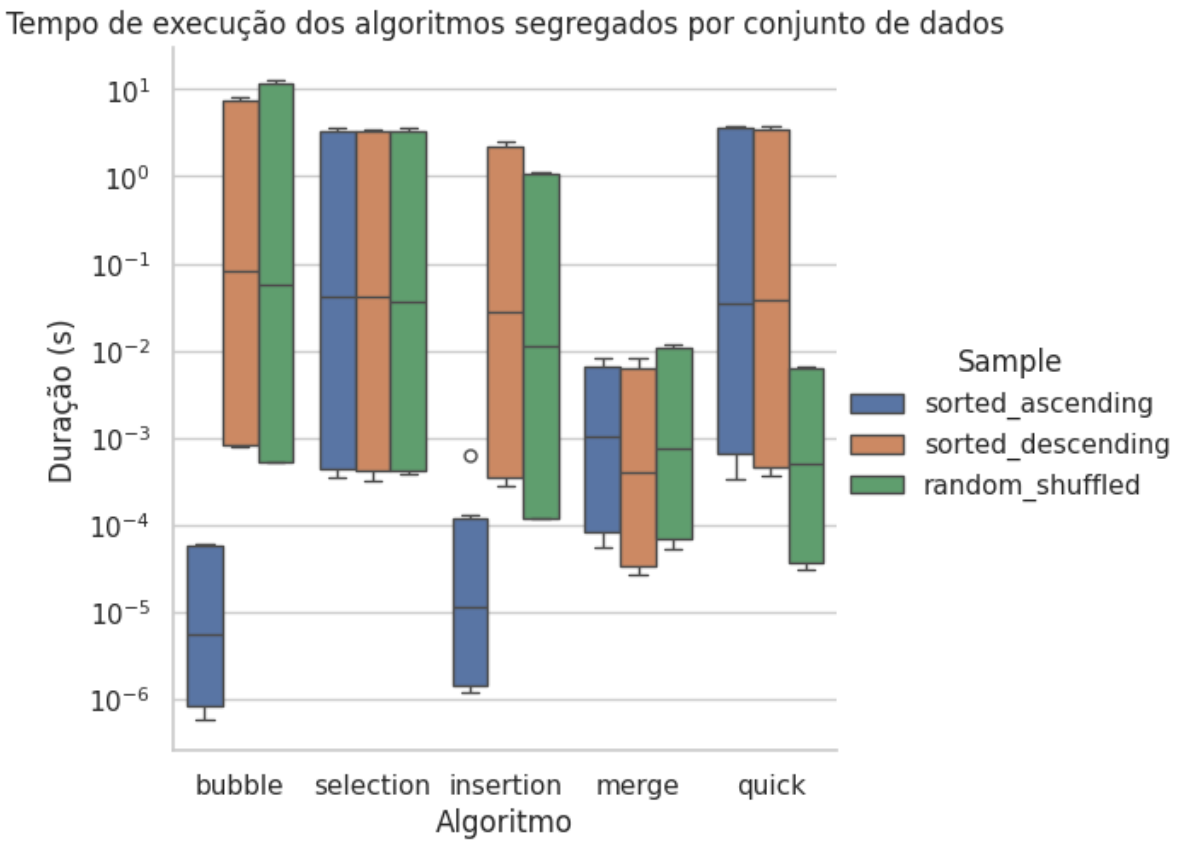

Figura 5 – Tempo de execução dos algoritmos segregados por conjunto de dados.

Discussão crítica

A comparação entre a análise assintótica dos algoritmos e os tempos medidos experimentalmente permite avaliar em que medida a teoria se confirma na prática e onde surgem divergências relevantes.

Algoritmos Quadráticos ((O(n^2)))

Nos algoritmos Bubble Sort, Selection Sort e Insertion Sort, a análise assintótica prevê crescimento quadrático do tempo de execução. Os resultados experimentais confirmam essa tendência: para entradas de 100.000 elementos, os tempos tornam-se inviáveis, chegando a várias ordens de grandeza acima dos algoritmos (O(n log n)). Entretanto, observa-se que:

- Insertion Sort apresenta tempos muito baixos em vetores já ordenados, aproximando-se de comportamento linear. Isso evidencia que a análise assintótica não captura totalmente a influência da estrutura inicial dos dados.
- Bubble Sort e Selection Sort mantêm desempenho consistentemente inferior, mesmo em cenários favoráveis, confirmando sua ineficiência prática.

Algoritmos Eficiêntes (O(n log n))

Merge Sort e Quick Sort confirmam a análise assintótica, mantendo tempos proporcionais a (n log n). Contudo, há nuances importantes:

- Merge Sort apresenta estabilidade em todos os cenários, refletindo fielmente sua complexidade teórica. Os tempos medidos mostram que, embora seja ligeiramente mais lento que Quick Sort em dados aleatórios, sua robustez o torna previsível e confiável.
- Quick Sort confirma sua eficiência média, mas nos cenários ordenados ou decrescentes os tempos se aproximam dos quadráticos. Isso demonstra que a análise assintótica média não reflete adequadamente o pior caso, que depende da escolha do pivô.

Influência da Estrutura dos Dados

Os resultados mostram que a análise assintótica, embora essencial para prever a escalabilidade, não é suficiente para explicar o desempenho prático em todos os cenários. A estrutura inicial dos dados exerce influência significativa:

- Vetores ordenados favorecem Insertion Sort, tornando-o competitivo em pequenas entradas.
- Vetores aleatórios favorecem Quick Sort, que se destaca em eficiência.
- Merge Sort mantém desempenho consistente, independentemente da entrada, confirmando sua robustez teórica e prática.

Conclusão

Os resultados obtidos confirmam que a análise assintótica é uma ferramenta fundamental para compreender a escalabilidade dos algoritmos, mas também revelam que fatores práticos — como a estrutura inicial dos dados e constantes ocultas — influenciam diretamente o desempenho real. Algoritmos quadráticos, como Bubble Sort, Selection Sort e Insertion Sort, mostraram-se inviáveis para grandes volumes de dados, embora o Insertion Sort tenha se destacado em cenários favoráveis, aproximando-se de comportamento linear.

Por outro lado, os algoritmos ($O(n \log n)$), Merge Sort e Quick Sort, mantiveram tempos significativamente menores e confirmaram sua superioridade teórica. Merge Sort demonstrou robustez e estabilidade em todos os cenários, enquanto Quick Sort foi altamente eficiente em entradas aleatórias, mas vulnerável em casos extremos, evidenciando a importância da estratégia de escolha do pivô.

Em síntese, conclui-se que a teoria assintótica fornece uma base sólida para prever o comportamento dos algoritmos, mas a escolha prática deve considerar também o contexto de aplicação. Para grandes conjuntos de dados, Merge Sort e Quick Sort são os mais indicados, enquanto Insertion Sort pode ser útil em vetores pequenos ou quase ordenados. Já Bubble Sort e Selection Sort devem ser restritos a fins didáticos, dada sua baixa eficiência prática.